SUPABASE

Complete Guide for HealthMate FYP

Zero → Hero Interview Preparation

Covers: What · Why · When · Where · How · SQL vs NoSQL · Pros & Cons
Architecture · Integration · Schema Design · Security · Interview Q&A

HealthMate — AI Preventive Healthcare Application
FYP Documentation · 2025

TABLE OF CONTENTS

1. **What is Supabase?** — Introduction, Architecture, Core Services
2. **Why Supabase? (Not MySQL)** — Motivation, Comparison, Decision Rationale
3. **SQL vs NoSQL — The Full Story** — Databases Explained, Where Supabase Sits
4. **Pros, Cons & Trade-offs** — Honest Evaluation for Production Use
5. **Supabase in HealthMate FYP** — How You Actually Used It End-to-End
6. **Database Schema Design** — Tables, Relationships, RLS, Indexes
7. **Authentication & Security** — Auth, RLS Policies, Encryption, HIPAA Alignment
8. **Real-time & Storage** — Live Health Data, File Storage, Wearables
9. **Integration Guide** — React/Next.js, Python ML, REST & Realtime APIs
10. **Mastering Supabase** — Learning Path, CLI, Edge Functions, Best Practices
11. **Interview Q&A;** — 30 Questions They Will Ask You

1 What is Supabase?

Introduction · Architecture · Core Services

Supabase is an open-source Backend-as-a-Service (BaaS) platform built on top of PostgreSQL. It is often described as the open-source alternative to Firebase. While Firebase uses a NoSQL document database, Supabase gives you a fully relational, SQL-powered PostgreSQL database — combined with everything else a modern application needs out of the box.

Think of Supabase as a bundle of powerful services, all pre-configured and talking to each other, so you don't need to wire them up yourself:

Service	What it Does	Used in HealthMate
PostgreSQL DB	Full relational SQL database	All health & user data
Auto REST API	PostgREST auto-generates REST from your schema	Fetch health records, risk scores
Authentication	Email, OAuth, magic link, phone OTP	User login & session management
Realtime	WebSocket subscriptions on DB changes	Live wearable data on dashboard
Storage	S3-compatible file storage with policies	Genetic reports, ECG files
Edge Functions	Serverless Deno functions at the edge	Trigger ML risk scoring
Row Level Security	Row-by-row access rules inside PostgreSQL	Patient data isolation

How Supabase Works Internally

Supabase is not a single monolithic service. It is a collection of open-source tools orchestrated together:

- **PostgreSQL** — the actual database engine. All your data lives here.
- **PostgREST** — reads your database schema and auto-generates a RESTful HTTP API. Zero code needed.
- **GoTrue** — handles authentication (JWT tokens, sessions, OAuth flows).
- **Realtime server** — listens to PostgreSQL WAL (Write-Ahead Log) and broadcasts changes over WebSockets.
- **Kong** — API gateway that routes all requests to the right service.
- **Supabase Studio** — a web dashboard for managing your database visually.

2 Why Supabase? (Not MySQL)

Motivation · Comparison · Decision Rationale

Your original FYP used MySQL. For the job role, you switched to Supabase. Here is the honest, structured reasoning for why Supabase is a better fit for HealthMate — not just as a resume decision, but as a genuine engineering choice.

MySQL vs Supabase — Direct Comparison

Feature	MySQL (Your Original)	Supabase
Database engine	MySQL	PostgreSQL (more powerful)
Authentication	Build it yourself	Built-in, production-ready
REST API	Build with Express/Django	Auto-generated — zero code
Realtime data	Poll manually or add Redis	Native WebSockets on DB changes
File storage	Set up S3 separately	Built-in with access policies
Row-level security	App-layer logic	Enforced inside the database
Dashboard / Admin UI	phpMyAdmin (basic)	Supabase Studio (modern, visual)
Scalability	Manual scaling	Managed, auto-scaling on cloud
Open source	Yes	Yes — self-host or cloud
Health data compliance	Manual setup	HIPAA-aligned controls built in
Setup time	Days (backend from scratch)	Minutes (one project creation)
SDK support	Driver libraries only	Official JS, Python, Flutter SDKs
Edge Functions	No	Yes (Deno — serverless)
Vector search (AI)	Not native	pgvector extension — native support

Specific Reasons for HealthMate

- **Health data is relational.** Users have profiles, risk scores, wearable readings, genetic markers, appointments. These are linked tables — not documents. PostgreSQL handles this perfectly.
- **Security is non-negotiable.** Patient data needs row-level access control. In MySQL you enforce this in your application code — one bug = data leak. Supabase enforces it inside the database via RLS policies.
- **Realtime dashboard.** HealthMate shows live wearable metrics. With MySQL you would poll every few seconds. Supabase gives you WebSocket subscriptions — the dashboard updates the moment data changes.

- **Faster development.** No time wasted building auth, API, file upload, or admin panel. Everything is ready — you focus on the AI and health logic.
- **AI/ML integration.** Supabase supports the pgvector extension for vector embeddings — useful when your ML models generate feature vectors for risk prediction.
- **Interview signal.** Supabase is used in production by thousands of startups. Knowing it shows awareness of modern full-stack tooling.

3 SQL vs NoSQL — The Full Story

Where Supabase Sits · Which to Use When

This is one of the most common interview topics. You need to explain it clearly, not just recite buzzwords.

SQL (Relational) Databases

SQL databases store data in structured tables with rows and columns. Relationships between tables are defined using foreign keys. The schema (structure) is defined before data is inserted.

- **Examples:** PostgreSQL, MySQL, SQLite, Oracle, SQL Server
- **Query language:** SQL — Structured Query Language
- **ACID compliant:** Atomicity, Consistency, Isolation, Durability — guarantees data integrity
- **Best for:** Financial systems, health records, e-commerce, any data with clear relationships
- **HealthMate relevance:** Users have profiles, wearable data, risk scores, genetic records — all relational

NoSQL Databases

NoSQL databases store data in flexible formats — documents (JSON), key-value pairs, graphs, or wide columns. Schema is flexible and can change per record.

- **Document DB examples:** MongoDB, Firestore (Firebase), CouchDB
- **Key-Value examples:** Redis, DynamoDB
- **Graph examples:** Neo4j
- **Best for:** Unstructured content, rapid prototyping, high write throughput, social graphs
- **HealthMate relevance:** Not ideal — patient data has fixed structure and requires ACID compliance

Where Supabase Sits

Aspect	SQL	NoSQL	Supabase (PostgreSQL)
Data structure	Tables/rows	Documents/JSON	Tables (+ JSON columns possible)
Schema	Strict, predefined	Flexible, dynamic	Strict + JSONB for flexibility
Relationships	Foreign keys, JOINs	Manual or embedded	Full JOIN support
ACID	Yes	Partial (varies)	Full ACID
Scaling	Vertical mainly	Horizontal easily	Vertical + read replicas
Query power	Very high (SQL)	Limited or custom	Very high (SQL + extensions)
Real-time	Not native	Firebase: yes	Yes (built-in)

Health data	Ideal	Risky (no ACID)	Ideal

Bottom line for your interview: HealthMate uses Supabase because health data is inherently relational, requires strict data integrity (ACID), and needs fine-grained access control per patient. NoSQL would make these things harder, not easier.

4 Pros, Cons & Trade-offs

Honest Evaluation — Not Marketing

Advantages of Supabase

- **PostgreSQL foundation:** The most feature-rich open-source relational database. Supports JSON, full-text search, geospatial, vector embeddings via pgvector.
- **Batteries included:** Auth, REST API, Realtime, Storage — all in one platform. Eliminates 60-70% of typical backend setup.
- **Open source & self-hostable:** No vendor lock-in. You can run Supabase on your own server if the cloud cost becomes prohibitive.
- **Row Level Security (RLS):** Security enforced at the database level — not in application code. Critical for multi-tenant health apps.
- **Real-time subscriptions:** Native WebSocket support. Dashboard updates live as health data changes without polling.
- **Auto-generated APIs:** PostgREST creates a full REST API from your schema. Add a table, get an endpoint — instantly.
- **Edge Functions:** Serverless Deno functions for running logic close to users. Useful for triggering ML risk scoring.
- **pgvector support:** Vector similarity search — enables storing ML model embeddings directly in the database.
- **Great developer experience:** Supabase Studio (dashboard), TypeScript SDK, excellent documentation, active community.
- **Generous free tier:** 500MB database, 1GB file storage, 50,000 monthly active users — enough to ship an FYP.

Disadvantages & Limitations

- **PostgreSQL learning curve:** If you only know MySQL, you need to learn PostgreSQL-specific syntax (e.g., SERIAL vs AUTO_INCREMENT, different string functions).
- **RLS complexity:** Row Level Security policies can be tricky to write correctly. A misconfigured policy can accidentally expose or block data.
- **Vendor dependency on cloud:** The managed cloud version introduces some dependency on Supabase's infrastructure. Self-hosting is complex for beginners.
- **Limited stored procedures UI:** Complex stored procedures are harder to manage visually compared to some enterprise MySQL tools.
- **Edge Functions are Deno only:** If your team knows Node.js only, Deno (for Edge Functions) has a slight learning curve.
- **Realtime has limits on free tier:** Maximum 200 concurrent realtime connections on the free plan — fine for FYP, not for large production apps.

- **Cold starts on Edge Functions:** Serverless functions can have latency on first invocation — not ideal for latency-sensitive medical alerts.
- **Complex migrations:** Schema migrations require careful management. Supabase has a migration tool (Supabase CLI) but it is less mature than Flyway or Liquibase.

5 Supabase in HealthMate FYP

How You Actually Used It — End to End

This chapter explains exactly how Supabase was integrated into HealthMate. This is what the interviewer wants to hear — specific, concrete usage, not generic descriptions.

Project Setup

- Created a Supabase project (cloud-hosted) — gets a PostgreSQL 15 database instantly.
- Installed the Supabase JavaScript/TypeScript SDK in the React frontend.
- Installed the Supabase Python client in the Flask/FastAPI ML backend.
- Used environment variables to store the Supabase URL and anon/service-role keys securely.
- Set up Supabase CLI for local development and schema migrations.

Installation commands used:

```
# Frontend (React/Next.js) npm install @supabase/supabase-js # Backend (Python/FastAPI) pip install supabase # CLI for migrations npm install -g supabase
```

Authentication in HealthMate

HealthMate uses Supabase Auth for user registration, login, and session management.

- **Email + password** registration for patients and healthcare providers.
- **JWT tokens** auto-managed by Supabase — no manual token handling.
- **Session persistence** in localStorage — user stays logged in across browser sessions.
- **Role-based access** — patient vs doctor vs admin differentiated via custom claims in the JWT.
- **OAuth** (Google login) added for convenience.

Auth code used in HealthMate:

```
// Initialize Supabase client import { createClient } from '@supabase/supabase-js' const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY) // Sign up a new patient const { data, error } = await supabase.auth.signUp({ email: 'patient@example.com', password: 'SecurePass123', options: { data: { role: 'patient', full_name: 'Ali Khan' } } }) // Sign in const { data, error } = await supabase.auth.signInWithPassword({ email: 'patient@example.com', password: 'SecurePass123' })
```

Data Operations — Health Records

How HealthMate reads and writes health data:

```
// Insert a new health reading from wearable const { data, error } = await supabase .from('health_readings') .insert({ user_id: user.id, heart_rate: 78, blood_glucose: 5.4, steps_today: 8432, recorded_at: new Date().toISOString() }) // Fetch last 7 days of readings for dashboard const { data } = await supabase .from('health_readings') .select('*') .eq('user_id', user.id) .gte('recorded_at',
```

```
sevenDaysAgo) .order('recorded_at', { ascending: false })
```

Realtime — Live Dashboard Updates

The HealthMate dashboard shows live wearable metrics. Using Supabase Realtime, the dashboard auto-updates when a new reading is inserted — no polling needed.

```
// Subscribe to new health readings for this user const channel = supabase
.channel('health-updates') .on('postgres_changes', { event: 'INSERT', schema:
'public', table: 'health_readings', filter: `user_id=eq.${user.id}` }, (payload) =>
{ // Update dashboard chart with new reading updateDashboard(payload.new) })
.subscribe()
```

Storage — Genetic Reports & Files

HealthMate allows users to upload genetic test reports (PDF) and medical documents. Supabase Storage handles this securely with access policies.

```
// Upload a genetic report PDF const { data, error } = await supabase.storage
.from('genetic-reports') .upload(`${user.id}/report_2025.pdf`, file, { contentType:
'application/pdf', upsert: false }) // Get a temporary signed URL (expires in 1
hour) const { data } = await supabase.storage .from('genetic-reports')
.createSignedUrl(`${user.id}/report_2025.pdf`, 3600)
```

Python ML Backend Integration

The ML risk prediction models (diabetes, obesity, cardiovascular) run in Python. They connect to Supabase to read health data and write back risk scores.

```
from supabase import create_client import os supabase =
create_client(os.environ['SUPABASE_URL'], os.environ['SUPABASE_SERVICE_KEY']) #
Read latest health readings for a patient response =
supabase.table('health_readings') \ .select('*') \ .eq('user_id', patient_id) \
.order('recorded_at', desc=True) \ .limit(30) \ .execute() readings = response.data
# After ML model runs, write risk scores back
supabase.table('risk_scores').upsert({ 'user_id': patient_id, 'diabetes_risk':
0.72, 'cardiovascular_risk': 0.45, 'obesity_risk': 0.31, 'calculated_at':
datetime.now().isoformat() }).execute()
```

6 Database Schema Design

Tables · Relationships · RLS · Indexes

This is the actual schema used in HealthMate. Every table has a clear purpose.

Table: users (managed by Supabase Auth)

- id (UUID, PK) — auto-created by Supabase Auth
- email, full_name, date_of_birth, gender
- role: ENUM('patient', 'doctor', 'admin')
- created_at TIMESTAMPTZ DEFAULT now()

Table: health_profiles

- id UUID PK, user_id UUID FK → users.id
- height_cm, weight_kg, blood_type
- existing_conditions TEXT[] (e.g. ['hypertension', 'diabetes'])
- family_history JSONB (flexible structure per condition)
- last_updated TIMESTAMPTZ

Table: health_readings

- id UUID PK, user_id UUID FK → users.id
- heart_rate INT, blood_glucose DECIMAL
- systolic_bp INT, diastolic_bp INT
- steps_today INT, sleep_hours DECIMAL
- calories_burned INT, bmi DECIMAL
- source VARCHAR (e.g. 'fitbit', 'apple_watch', 'manual')
- recorded_at TIMESTAMPTZ

Table: genetic_markers

- id UUID PK, user_id UUID FK → users.id
- gene_name VARCHAR, variant VARCHAR, risk_allele VARCHAR
- associated_condition VARCHAR, risk_level VARCHAR
- report_file_path TEXT (Supabase Storage path)
- uploaded_at TIMESTAMPTZ

Table: risk_scores

- id UUID PK, user_id UUID FK → users.id
- diabetes_risk DECIMAL(4,3), cardiovascular_risk DECIMAL(4,3)

- obesity_risk DECIMAL(4,3), overall_risk DECIMAL(4,3)
- risk_factors JSONB (which factors drove the score)
- model_version VARCHAR, calculated_at TIMESTAMPTZ

Table: recommendations

- id UUID PK, user_id UUID FK → users.id
- category VARCHAR (diet / exercise / medication / checkup)
- title TEXT, description TEXT, priority INT
- is_completed BOOLEAN DEFAULT false
- generated_at TIMESTAMPTZ, expires_at TIMESTAMPTZ

Table: appointments

- id UUID PK, patient_id FK → users.id, doctor_id FK → users.id
- scheduled_at TIMESTAMPTZ, duration_minutes INT
- status ENUM('scheduled','completed','cancelled')
- notes TEXT, created_at TIMESTAMPTZ

Key SQL for HealthMate

```
-- Create health_readings table CREATE TABLE health_readings ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE, heart_rate INT CHECK (heart_rate > 0 AND heart_rate < 300), blood_glucose DECIMAL(5,2), systolic_bp INT, diastolic_bp INT, steps_today INT DEFAULT 0, source VARCHAR(50) DEFAULT 'manual', recorded_at TIMESTAMPTZ NOT NULL DEFAULT now() ); -- Index for fast user-based queries CREATE INDEX idx_readings_user_time ON health_readings(user_id, recorded_at DESC); -- Enable RLS ALTER TABLE health_readings ENABLE ROW LEVEL SECURITY;
```

7 Authentication & Security

Auth · RLS Policies · Encryption · Privacy

Security is critical in a healthcare application. HealthMate handles sensitive personal health data, genetic information, and medical records. Supabase provides multiple layers of security that were all used in the project.

Row Level Security (RLS) — Most Important Concept

RLS is the feature that makes Supabase uniquely secure for multi-user health apps. Instead of checking 'does this user own this record?' in your React/Python code, you write a policy directly in PostgreSQL. The database itself enforces it — no matter how the data is queried.

RLS policies used in HealthMate:

```
-- Users can only read their own health readings CREATE POLICY
patient_readings_select ON health_readings FOR SELECT USING (auth.uid() = user_id);
-- Users can only insert their own readings CREATE POLICY patient_readings_insert
ON health_readings FOR INSERT WITH CHECK (auth.uid() = user_id); -- Doctors can
read their patients' readings CREATE POLICY doctor_readings_select ON
health_readings FOR SELECT USING ( EXISTS ( SELECT 1 FROM appointments WHERE
doctor_id = auth.uid() AND patient_id = health_readings.user_id AND status =
'scheduled' ) ); -- Admins can read everything CREATE POLICY admin_all_access ON
health_readings FOR ALL USING (auth.jwt() ->> 'role' = 'admin');
```

Security Layers in HealthMate

Security Layer	How Implemented	What it Protects
Transport encryption	HTTPS / TLS 1.3 (Supabase default)	Data in transit
Database encryption	AES-256 at rest (Supabase managed)	Stored patient data
Auth tokens	JWT with short expiry + refresh tokens	Session hijacking
Row Level Security	PostgreSQL policies (per table)	Cross-user data access
Storage policies	Supabase Storage RLS	Genetic report files
API keys	Anon key (client) vs Service key (server)	Backend privilege escalation
Environment vars	.env files, never committed to Git	Key exposure

API Key Security — Critical Point

Supabase has TWO keys — you must understand both for the interview:

- **Anon key** — safe to use in frontend code. Limited by RLS policies. If someone steals it, RLS still protects your data.

- **Service role key** — bypasses ALL RLS. Never expose this in frontend. Only used in your Python ML backend (server-side). Store in environment variables only.

8 Real-time & Storage

Live Health Data · File Storage · Wearables

How Realtime Works in Supabase

Supabase Realtime works by listening to PostgreSQL's Write-Ahead Log (WAL). When any INSERT, UPDATE, or DELETE happens in a table, the Realtime server broadcasts that change to all WebSocket clients subscribed to that table.

- No polling required — the dashboard updates instantly when wearable data arrives.
- Filter subscriptions by user_id so each patient only receives their own updates.
- Three event types: INSERT (new reading), UPDATE (edited record), DELETE (removed).
- Channels are the WebSocket connections — one channel per table subscription.
- Presence feature: know which users are currently online (useful for doctor-patient sessions).

Storage for Genetic & Medical Files

HealthMate uses Supabase Storage for genetic test reports (PDFs), ECG images, and medical documents. Storage is S3-compatible and integrates with RLS.

```
// Storage bucket setup (in Supabase Dashboard or SQL) -- Create a private bucket (not publicly accessible) INSERT INTO storage.buckets (id, name, public) VALUES ('genetic-reports', 'genetic-reports', false); -- Policy: Users can only access their own folder CREATE POLICY user_own_files ON storage.objects FOR ALL USING (auth.uid()::text = (storage.foldername(name))[1]);
```

9 Integration Guide

React · Python · REST API · Best Practices

Full Setup in React (Frontend)

```
// src/lib/supabase.js — Single client instance import { createClient } from '@supabase/supabase-js' const supabaseUrl = process.env.REACT_APP_SUPABASE_URL const supabaseAnonKey = process.env.REACT_APP_SUPABASE_ANON_KEY export const supabase = createClient(supabaseUrl, supabaseAnonKey, { auth: { persistSession: true, autoRefreshToken: true } })
```

Full Setup in Python (ML Backend)

```
# ml_backend/db.py import os from supabase import create_client, Client # Use SERVICE KEY for backend — bypasses RLS for ML processing supabase: Client = create_client( os.environ['SUPABASE_URL'], os.environ['SUPABASE_SERVICE_KEY'] # NEVER expose this in frontend ) def get_patient_data(patient_id: str) -> dict: profile = supabase.table('health_profiles') \ .select('*, genetic_markers(*)') \ .eq('user_id', patient_id) \ .single() \ .execute() readings = supabase.table('health_readings') \ .select('*') \ .eq('user_id', patient_id) \ .order('recorded_at', desc=True) \ .limit(90) \ .execute() return { 'profile': profile.data, 'readings': readings.data }
```

Useful Query Patterns

Goal	Supabase Query
Get all user data (JOIN)	.select('*, health_profiles(*), risk_scores(*)')
Filter by date range	.gte('recorded_at', fromDate).lte('recorded_at', toDate)
Get single record	.eq('id', recordId).single()
Insert and return new row	.insert({...}).select().single()
Update a record	.update({status: 'done'}).eq('id', appointmentId)
Delete a record	.delete().eq('id', recordId)
Count rows	.select('*', { count: 'exact', head: true })
Full-text search	.textSearch('notes', 'diabetes risk')
Order + paginate	.order('created_at', {asc: false}).range(0, 19)

10 Mastering Supabase

Learning Path · CLI · Edge Functions · Tips

Learning Path (Fastest Route)

- **Day 1-2:** Read docs.supabase.com — quickstart guide. Create a project. Run basic CRUD queries in the Studio Table Editor.
- **Day 3-4:** Set up Auth. Build email login + logout. Understand JWT and session handling.
- **Day 5-6:** Write RLS policies. Test them using the Supabase Studio policy tester.
- **Day 7-8:** Add Realtime subscription in your frontend. Watch the database and UI sync live.
- **Day 9-10:** Set up Supabase CLI. Write a migration. Understand how schema versioning works.
- **Day 11-12:** Write an Edge Function. Trigger it from a database webhook.
- **Day 13+:** Learn pgvector for ML embedding storage. Study PostgREST advanced filtering.

Supabase CLI — Key Commands

```
# Install and login npx supabase login # Initialize a project locally npx supabase init # Start local Supabase (Docker required) npx supabase start # Create a new migration npx supabase migration new add_risk_scores_table # Apply migrations to production npx supabase db push # Generate TypeScript types from your schema npx supabase gen types typescript --linked
```

Edge Functions for ML Triggers

Edge Functions are serverless Deno functions. In HealthMate, they are used to trigger ML risk recalculation when a new health reading is inserted.

```
// supabase/functions/calculate-risk/index.ts import { serve } from 'https://deno.land/std@0.168.0/http/server.ts' serve(async (req) => { const { user_id } = await req.json() // Call your Python ML API const response = await fetch(`${ML_API_URL}/calculate-risk`, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ user_id }) }) const risk = await response.json() return new Response(JSON.stringify(risk), { headers: { 'Content-Type': 'application/json' } }) })
```

11 Interview Q&A

30 Questions They Will Ask — With Strong Answers

These are the actual questions likely in your interview. Each answer is framed specifically around HealthMate so it sounds natural and experience-based.

Q1: What is Supabase and why did you choose it?

Supabase is an open-source BaaS built on PostgreSQL. I chose it for HealthMate because health data is relational — patients have profiles, readings, genetic markers, and risk scores that are all linked. PostgreSQL handles this cleanly with foreign keys and JOINs. On top of that, Supabase gives me authentication, a real-time layer, file storage, and row-level security out of the box — things I would have spent weeks building from scratch with MySQL.

Q2: What is the difference between Supabase and Firebase?

Firebase uses Firestore, which is a NoSQL document database — great for unstructured data but weak on complex queries and ACID compliance. Supabase uses PostgreSQL — fully relational, ACID-compliant, supports JOINs and transactions. For health data that has strict relationships and requires data integrity guarantees, Supabase is the better fit. Firebase is faster to prototype with, but Supabase is more robust for structured, sensitive data.

Q3: What is Row Level Security and how did you use it?

RLS is a PostgreSQL feature that enforces access control at the database level, not the application level. I wrote policies on every sensitive table — health_readings, risk_scores, genetic_markers. The policy on health_readings says: a user can only SELECT rows where user_id = auth.uid(). This means even if someone bypasses my frontend, the database itself refuses to return another patient's data.

Q4: What is the difference between the anon key and the service key?

The anon key is used in the frontend. It is safe to expose because all queries made with it are still restricted by RLS policies. The service key bypasses RLS entirely — it is admin-level access. I only used the service key in my Python ML backend, stored as an environment variable, never committed to Git or sent to the frontend.

Q5: How did you handle real-time data from wearables?

I used Supabase Realtime. The dashboard subscribes to INSERT events on the health_readings table, filtered by the current user's ID. When a wearable pushes a new reading, it does an insert into the table. Supabase detects that via the PostgreSQL WAL and broadcasts it over WebSocket to the dashboard. The chart updates instantly without any polling.

Q6: Why PostgreSQL over MySQL for this project?

PostgreSQL has several advantages for HealthMate: it supports JSONB for flexible genetic data structures, has the pgvector extension for storing ML embeddings, supports Row Level Security natively, and has more powerful indexing options like partial indexes and GIN indexes for JSONB. MySQL lacks native RLS, and its JSON support is weaker. PostgreSQL is also what Supabase is built on, so the integration is seamless.

Q7: How did you store genetic data which is semi-structured?

I used PostgreSQL's JSONB column type in the health_profiles table for family_history, which varies per patient. For structured genetic markers (gene name, variant, risk allele), I used a separate genetic_markers table with fixed columns. JSONB lets you query inside the JSON using PostgreSQL operators, which is more powerful than storing raw text JSON.

Q8: What is PostgREST and how did it help you?

PostgREST is the component inside Supabase that reads your database schema and automatically generates a RESTful API. When I created the health_readings table, Supabase immediately gave me endpoints to GET, POST, PATCH, and DELETE readings — with filtering, ordering, and pagination supported through query parameters. I did not write any Express routes for standard CRUD operations.

Q9: How did you integrate Supabase with your ML model?

The ML models run in a Python FastAPI service. It uses the supabase-py client with the service key to read patient health readings, genetic markers, and profile data. It processes this through scikit-learn models trained on health datasets to produce risk probabilities. The risk scores are then written back to Supabase's risk_scores table. The frontend subscribes to changes on that table, so the risk dashboard updates automatically when the ML pipeline finishes.

Q10: What challenges did you face with Supabase?

The biggest challenge was writing RLS policies correctly. If a policy is wrong, queries either silently return empty results or accidentally expose data. I had to use the Supabase Studio policy editor and test with different user contexts to verify each policy. Another challenge was understanding when to use the anon key vs service key in the Python backend, and ensuring the service key was never exposed.

Q11: What is a Supabase Edge Function? Did you use one?

Edge Functions are serverless Deno functions that run at Supabase's infrastructure, close to users. I used one as a webhook trigger — when a new health reading is inserted, a database webhook fires the Edge Function, which calls my Python ML API to recalculate the patient's risk scores. This decouples the real-time data pipeline from the frontend.

Q12: How does Supabase handle HIPAA or health data compliance?

Supabase on its Business/Enterprise plan offers HIPAA Business Associate Agreements. The platform uses AES-256 encryption at rest and TLS in transit. Combined with RLS policies for data isolation, short-lived JWTs for sessions, and signed URLs for medical file access, the security model aligns with HIPAA's requirements for PHI protection. For production, I would enable audit logging and use the enterprise plan.

Q13: How is data migration handled in Supabase?

Supabase uses SQL migration files managed through the Supabase CLI. Each schema change is a numbered .sql file in the supabase/migrations folder. Running 'supabase db push' applies pending migrations to production. This gives you version-controlled schema history, which is important for a team project where multiple people might change the database.

Q14: Explain SQL joins and give an example from HealthMate.

A JOIN combines rows from two tables based on a related column. In HealthMate, to get a patient's latest risk score along with their profile: SELECT u.full_name, hp.blood_type, rs.diabetes_risk FROM users u JOIN health_profiles hp ON u.id = hp.user_id JOIN risk_scores rs ON u.id = rs.user_id WHERE u.id = 'patient-uuid' ORDER BY rs.calculated_at DESC LIMIT 1. This gives a complete patient health summary in one query.

Q15: What is the difference between JSONB and JSON in PostgreSQL?

JSON stores the text as-is, meaning every query must re-parse it. JSONB stores it in a decomposed binary format, which is faster to query and supports indexing with GIN indexes. In HealthMate's health_profiles table, family_history is JSONB so I can query like: SELECT * FROM health_profiles WHERE family_history @> '{"diabetes": true}' — this works fast even with thousands of records because the JSONB column can be indexed.

Quick-Fire Concepts to Know

Term	One-line Definition
WAL (Write-Ahead Log)	PostgreSQL log that records all DB changes — Realtime reads this
JWT	JSON Web Token — signed token Supabase uses for auth sessions
auth.uid()	Supabase function that returns the current logged-in user's ID inside RLS polic
gen_random_uuid()	PostgreSQL function for generating UUID primary keys
PostgREST	The component that turns your schema into a REST API automatically
pgvector	PostgreSQL extension for storing AI/ML vector embeddings
ACID	Atomicity, Consistency, Isolation, Durability — guarantees of relational DBs
Bucket (Storage)	A container in Supabase Storage — like a folder with access policies

Channel (Realtime)	A WebSocket connection subscribed to changes in a specific table/row
Migration	A versioned SQL file that modifies the database schema
Deno	JavaScript/TypeScript runtime used by Supabase Edge Functions

Final Note: You switched from MySQL to Supabase. In your interview, frame this as a deliberate, reasoned engineering decision — not just a resume line. Say: 'I evaluated the requirements of a healthcare app — relational data, strict security, real-time dashboard, and rapid development — and Supabase satisfied all four better than a bare MySQL setup would have.' That answer shows judgment, not just tool knowledge.